PROJECT DOCUMENTATION

FitBuddy

AI Fitness Plan Generator using Google Gemini Models

Built with FastAPI | SQLAlchemy | SQLite | Google Gemini

Submitted to: Government of Tamil Nadu

Submitted by: HARISIVAN R

Date: 30 September 2026

Table of Contents

1. Overview
2. Key Features
3. Tech Stack
4. Architecture
5. Project Structure
6. Setup and Installation
7. API Endpoints
8. Database Schema
9. Future Improvements

1. Overview

FitBuddy is an AI-powered web application that creates personalised fitness plans. A user enters basic details such as name, age, height, weight, fitness goal and preferred workout intensity. The application sends these details to Google's Gemini generative AI models and returns a complete 7-day workout plan together with a nutrition and recovery tip that matches the user's goal.

If the user is not happy with any part of the plan, they can type feedback in plain language (for example, "add more cardio" or "include yoga"). FitBuddy sends the original plan and the feedback back to Gemini, which rewrites the whole plan. Every user and every plan is saved in a local SQLite database, and an admin dashboard lists all stored records.

1.1 Problem Statement

Generic workout charts do not consider a person's age, body weight, goal or comfort level. Personal trainers are useful but are not affordable or accessible to everyone. FitBuddy addresses this by giving anyone with a web browser an instant, personalised and adjustable plan.

1.2 Objectives

- Generate a personalised 7-day workout plan (warm-up, main exercises with sets and reps, cool-down) from user inputs.
- Provide a goal-specific nutrition and recovery tip along with the plan.
- Let users refine the plan using natural-language feedback.
- Store user profiles and plans persistently for later review.
- Keep the application working even when the AI service is rate-limited or unavailable.

2. Key Features

Feature	Description
Personalised 7-day plan	Uses age, weight, goal and intensity to build a day-by-day plan. Each day contains a warm-up, main exercises with sets/reps and cool-down tips.
Nutrition & recovery tip	A short, goal-specific nutrition tip is generated in the same AI call as the workout plan.
AI feedback loop	Users submit feedback such as "add more cardio"; Gemini rewrites the complete plan to include it and the result is stored as the updated plan.
Markdown rendering	Gemini returns Markdown, which the browser converts into formatted headings and lists using the marked.js library.
Admin dashboard	The /view-all-users page shows every user with their profile, original plan and updated plan in a table.
Data persistence	Users and plans are saved with SQLAlchemy ORM in a SQLite database (fitbuddy.db). Returning users are updated instead of duplicated.
Automatic model selection	At start-up the app asks the API which Gemini models are available and picks the first supported one (gemini-2.5-flash, then gemini-2.0-flash, then gemini-flash-latest).
Retry with back-off	On rate-limit errors (HTTP 429) the request is retried up to 3 times, waiting 2, 4 and 8 seconds between attempts as configured.
Fallback content	If the AI is rate-limited or the model is not found, a built-in 7-day backup plan and a generic nutrition tip are shown so the user never sees a blank page.
Responsive dark UI	A custom dark-theme interface built with plain HTML and CSS (about 290 lines) with a navigation bar, cards and a scrollable table.

3. Tech Stack

Layer	Technology	Purpose
Language	Python	Application and business logic
Web framework	FastAPI 0.135.1	Routing, form handling, dependency injection, auto-generated API docs at /docs
Server	Uvicorn 0.41.0	ASGI server with live reload for development
Templating	Jinja2 3.1.6	Server-side HTML pages (index, result, all_users)
Frontend	HTML, CSS, JavaScript	Custom styling and marked.js (CDN) to render Markdown
Database	SQLite	Lightweight file-based storage (fitbuddy.db)
ORM	SQLAlchemy 2.0.48	Maps Python classes to database tables
Validation	Pydantic 2.12.5	Data schemas (UserSchema, WorkoutPlanSchema)
AI SDK	google-generativeai 0.8.6	Calls the Gemini models
AI models	Gemini 2.5 Flash / 2.0 Flash	Workout plan generation, nutrition tip and plan updates
Configuration	python-dotenv 1.2.2	Loads GOOGLE_API_KEY from the .env file

Note: The README mentions Gemini 1.5 Pro and 1.5 Flash. The current source code selects Gemini 2.5 Flash and 2.0 Flash, so those are the models documented here.

4. Architecture

FitBuddy follows a simple layered architecture. The browser talks to a FastAPI application. The route layer coordinates two back-end services: the AI layer that calls Google Gemini, and the database layer that stores data in SQLite through SQLAlchemy.

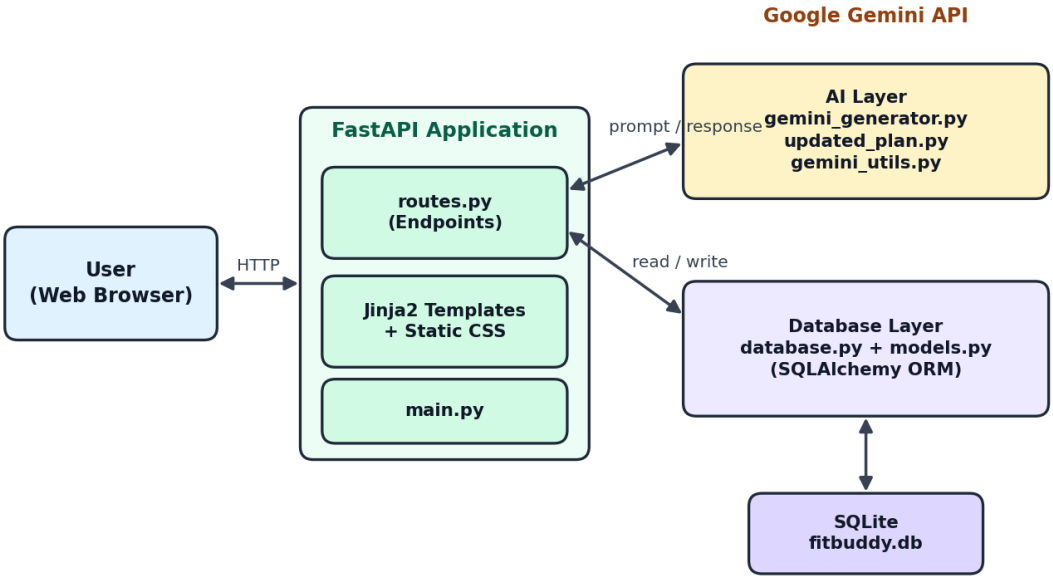


Figure 1: FitBuddy system architecture

4.1 Layers

- **Presentation layer:** Jinja2 templates (index.html, result.html, all_users.html) and static/style.css.
- **Application layer:** app/main.py creates the FastAPI app, creates database tables at start-up, mounts /static and includes the router from app/routes.py.
- **AI layer:** gemini_generator.py (plan + tip), updated_plan.py (feedback rewrite), gemini_flash_generator.py (stand-alone tip) and gemini_utils.py (model discovery, retry, error detection).
- **Data layer:** database.py (engine, session, helper functions) and models.py (ORM classes).

4.2 Request Flow: Generating a Plan

1. **Submit:** The user fills the form on the home page and submits it to POST /generate-workout.
2. **Generate:** routes.py calls generate_plan_and_tip(age, weight, goal, intensity). One prompt asks Gemini for a JSON object with workout_plan and nutrition_tip.
3. **Protect:** gemini_utils retries on rate limits. If the AI still fails, a fallback plan is returned.
4. **Save:** The user is looked up by user_id. A new user is inserted, or an existing user's details are updated. A new WorkoutPlan row is then saved.
5. **Display:** result.html is rendered; the browser converts the Markdown to formatted HTML.

4.3 Request Flow: Updating a Plan with Feedback

1. **Submit:** The user types feedback and the form posts wp_id and feedback to POST /submit-feedback.
2. **Load:** The original plan is read from the database using wp_id.
3. **Rewrite:** get_updated_plan(original_plan, feedback) asks Gemini to rewrite the full 7-day plan with the change.
4. **Store:** The result is saved in the updated_plan column and shown to the user.

5. Project Structure

```
FitBuddy-AI-Fitness-Plan-Generator-using-Gemini-Models-main/
|-- app/
|   |-- main.py                  # FastAPI app, DB setup, static files
|   |-- routes.py                # HTTP endpoints
|   |-- database.py              # Engine, session, CRUD helper functions
|   |-- models.py                # User and WorkoutPlan ORM models
|   |-- schemas.py               # Pydantic schemas
|   |-- gemini_generator.py      # Workout plan + nutrition tip generation
|   |-- gemini_flash_generator.py # Stand-alone nutrition tip generator
|   |-- updated_plan.py          # Feedback-based plan rewrite
|   `-- gemini_utils.py          # Model selection, retry, error helpers
|-- templates/
|   |-- index.html               # Input form (home page)
|   |-- result.html              # Plan, tip and feedback form
|   `-- all_users.html           # Admin dashboard table
|-- static/
|   `-- style.css                # Dark theme styling
|-- requirements.txt             # Python dependencies
|-- .env.example                 # Sample environment file
|-- .gitignore
`-- README.md
```

5.1 File Descriptions

File	Responsibility
app/main.py	Creates the FastAPI instance, runs Base.metadata.create_all to create tables, serves /static and registers the router.
app/routes.py	Defines the four page routes and coordinates AI calls, database saves and template rendering.
app/database.py	SQLite connection (sqlite:///./fitbuddy.db), get_db session dependency, and save_user, save_plan, update_plan, get_user, get_original_plan and get_all_users.
app/models.py	SQLAlchemy models User and WorkoutPlan with a one-to-many relationship.
app/schemas.py	Pydantic models UserSchema and WorkoutPlanSchema (prepared for API responses).

File	Responsibility
app/gemini_generator.py	Builds the main prompt, requests JSON output, parses it and provides the backup 7-day plan.
app/gemini_flash_generator.py	Generates a nutrition tip on its own. Currently not called by the routes because the tip is produced together with the plan.
app/updated_plan.py	Sends the original plan and user feedback to Gemini and returns the rewritten plan.
app/gemini_utils.py	build_model (discover available model), generate_with_retry (exponential back-off), is_rate_limit_error, is_model_not_found_error.

6. Setup and Installation

6.1 Prerequisites

- Python 3.9 or higher (3.10+ recommended) and pip.
- A Google Gemini API key. A free key can be created in Google AI Studio (aistudio.google.com).
- A terminal (Command Prompt, PowerShell, Terminal) and a modern web browser. Internet access is needed for the AI calls and for loading marked.js.

6.2 Installation Steps

1. **Open the project folder** in a terminal:

```
cd FitBuddy-AI-Fitness-Plan-Generator-using-Gemini-Models-main
```

2. **Create a virtual environment:**

```
python -m venv venv
```

3. **Activate it:**

```
# Windows
venv\Scripts\activate

# macOS / Linux
source venv/bin/activate
```

4. **Install dependencies:**

```
pip install -r requirements.txt
```

5. **Add the API key.** Copy .env.example to .env and put your key inside:

```
GOOGLE_API_KEY=your_actual_key_here
```

6. **Start the server:**

```
uvicorn app.main:app --reload
```

7. **Open the application** at http://127.0.0.1:8000. Interactive API documentation is available at http://127.0.0.1:8000/docs.

6.3 Troubleshooting

Problem	Solution
Plan shows "Error generating plan"	Check that GOOGLE_API_KEY is set correctly in .env and that the server was restarted after editing it.
Fallback plan / "retry in 1-2 minutes"	The API quota was exceeded (HTTP 429). Wait a minute and try again.
pip cannot read requirements.txt	The file is saved in UTF-16 encoding. Re-save it as UTF-8 and run pip install again.
Markdown appears as plain text	The page could not load marked.js from the CDN. Check the internet connection.
Database looks stale after code changes	Stop the server and delete fitbuddy.db; tables are re-created on the next start.

7. API Endpoints

All endpoints return server-rendered HTML pages. Form fields are sent as application/x-www-form-urlencoded.

Method	Endpoint	Description	Returns
GET	/	Home page with the user details form.	index.html
POST	/generate-workout	Generates the 7-day plan and nutrition tip, saves the user and plan.	result.html
POST	/submit-feedback	Rewrites an existing plan using the user's feedback and saves it.	result.html (or 404 text)
GET	/view-all-users	Admin table of all users and their plans.	all_users.html
GET	/static/{file}	Serves static assets such as style.css.	CSS file
GET	/docs	Auto-generated Swagger UI (built into FastAPI).	Swagger UI

7.1 POST /generate-workout: Form Fields

Field	Type	Description
username	string	Display name of the user
user_id	string	Unique ID chosen by the user; an existing ID updates that user's profile
age	integer	Age in years

Field	Type	Description
height	integer	Height in cm
weight	integer	Body weight
goal	string	Fitness goal, for example "Build Muscle" or "Lose Fat"
intensity	string	low, medium or high

7.2 POST /submit-feedback: Form Fields

Field	Type	Description
wp_id	integer	ID of the workout plan to update (sent as a hidden field)
feedback	string	Free-text change request, for example "add more cardio"

7.3 Error Handling

- If wp_id does not exist, /submit-feedback returns HTTP 404 with the text "Error: Workout plan not found."
- Rate-limit or model errors from Gemini do not crash the page; a backup plan or a note is returned instead.
- Missing or non-numeric form fields are rejected by FastAPI with HTTP 422.

8. Database Schema

FitBuddy uses SQLite through SQLAlchemy. The database file fitbuddy.db is created automatically in the project root when the server first starts. There are two tables with a one-to-many relationship: one user can have many workout plans.

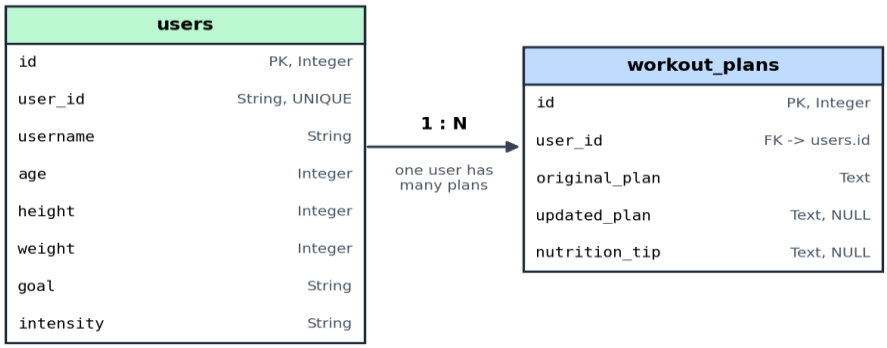


Figure 2: Entity relationship diagram

8.1 Table: users

Column	Type	Constraint	Description
`id`	Integer	Primary key, indexed	Internal row ID
`user_id`	String	Unique, indexed	User-chosen ID from the form
`username`	String	Indexed	Name of the user
`age`	Integer	-	Age in years
`height`	Integer	-	Height
`weight`	Integer	-	Weight
`goal`	String	-	Fitness goal
`intensity`	String	-	low / medium / high

8.2 Table: workout_plans

Column	Type	Constraint	Description
`id`	Integer	Primary key, indexed	Plan ID (used as wp_id)
`user_id`	Integer	Foreign key to users.id	Owner of the plan
`original_plan`	Text	-	Plan generated by Gemini (Markdown)

Column	Type	Constraint	Description
updated_plan	Text	Nullable	Plan after feedback; empty until feedback is given
nutrition_tip	Text	Nullable	Nutrition and recovery tip (Markdown)

8.3 Relationship

In models.py, User.workout_plans and WorkoutPlan.user are linked with SQLAlchemy relationship() and back_populates. This lets the admin page loop through user.workout_plans directly in the template.

9. Future Improvements

Improvement	Benefit
User login and admin protection	Add authentication so that /view-all-users is visible only to administrators and users can see only their own plans.
Tamil and multi-language support	Let users choose Tamil or other languages for plans and tips so that more people in Tamil Nadu can use the application.
Plan history and versions	Feedback currently rewrites the original plan each time and only one updated plan is kept. Storing each version would allow repeated refinement and undo.
Richer health profile	Collect dietary preference (vegetarian, non-vegetarian), medical conditions, equipment available and available time, and include them in the prompt. Offer kg/lbs and cm units explicitly.
Complete meal plans	Extend the nutrition tip into a weekly Indian-diet meal plan with calories and macros.
Progress tracking	Let users log weight and workouts and show charts of their progress.
Export and sharing	Download the plan as PDF or share it through WhatsApp or email.
REST/JSON API	Use the existing Pydantic schemas to expose JSON endpoints so that a mobile app can use the same back end.
Security hardening	Sanitise the Markdown output before inserting it into the page, add input limits, and keep API keys only in environment variables.
Performance	The Gemini calls are blocking; running them in a thread pool or with the async client will keep the server responsive under load. Caching common plans would reduce API usage.
Production database and deployment	Move from SQLite to PostgreSQL, use Alembic migrations, and package the application in Docker for cloud deployment.
Testing and code clean-up	Add unit tests for the routes and AI helpers, remove the unused gemini_flash_generator.py or use it in the routes, and show height on the result page after feedback.

Conclusion

FitBuddy demonstrates how generative AI can make personalised fitness guidance available to everyone through a simple web application. Its modular FastAPI structure, persistent storage and resilient AI handling provide a solid base for the improvements above.